Lesson 7 Worksheet: The Challenge of Turning — ANSWER KEY

Part 1: Direction from Coordinates

Current Position	Next Position	row_diff	col_diff	Direction
(0, 0)	(1, 0)	1 - 0 = 1	0 - 0 = 0	S
(2, 3)	(1, 3)	1 - 2 = -1	3 - 3 = 0	N
(1, 1)	(1, 2)	0	1	E
(3, 2)	(3, 1)	0	-1	W
(0, 3)	(1, 3)	1	0	S
(2, 0)	(2, 1)	0	1	E
(3, 1)	(2, 1)	-1	0	N
(0, 2)	(0, 1)	0	-1	W

Part 2: The Turn Table

Current Heading	Need N	Need S	Need E	Need W
N	None	180	Right 90	Left 90
S	180	None	Left 90	Right 90
E	Left 90	Right 90	None	180
W	Right 90	Left 90	180	None

- How many cells say "None"? 4 — Why? Each heading already faces one of the four directions, so when the needed direction matches the current heading, no turn is needed. The 4 "None" cells fall along the diagonal.
- How many cells say "180"? 4 — What pattern? They are always opposite pairs: N↔S and E↔W. The 180 cells also fall along the anti-diagonal.

Part 3: Determine the Turn

Scenario	Current Heading	Needed Direction	Turn
1	N	E	Right 90
2	E	E	None
3	S	N	180
4	W	S	Left 90
5	E	N	Left 90
6	N	W	Left 90
7	S	W	Right 90
8	W	E	180

Part 4: Trace a Path with Heading Updates

Path: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3)], Starting heading: North

Step	From	To	Needed Direction	Current Heading	Turn	New Heading
1	(0, 0)	(1, 0)	S	N	180	S
2	(1, 0)	(2, 0)	S	S	None	S
3	(2, 0)	(2, 1)	E	S	Left 90	E
4	(2, 1)	(2, 2)	E	E	None	E
5	(2, 2)	(2, 3)	E	E	None	E

- How many turns were needed? 2 (the 180 at step 1 and the left turn at step 3)
- How many times was no turn needed? 3 (steps 2, 4, 5)
- At which step did the robot change from row to column movement? Step 3

Part 5: Trace a Second Path

Path: [(2, 3), (1, 3), (0, 3), (0, 2), (0, 1), (0, 0)], Starting heading: South

Step	From	To	Needed Direction	Current Heading	Turn	New Heading
1	(2, 3)	(1, 3)	N	S	180	N
2	(1, 3)	(0, 3)	N	N	None	N
3	(0, 3)	(0, 2)	W	N	Left 90	W
4	(0, 2)	(0, 1)	W	W	None	W
5	(0, 1)	(0, 0)	W	W	None	W

Final heading after completing path: W

Part 6: Trace a Third Path

Path: [(1, 1), (1, 2), (1, 3), (2, 3), (3, 3)], Starting heading: North

Step	From	To	Needed Direction	Current Heading	Turn	New Heading
1	(1, 1)	(1, 2)	E	N	Right 90	E
2	(1, 2)	(1, 3)	E	E	None	E
3	(1, 3)	(2, 3)	S	E	Right 90	S
4	(2, 3)	(3, 3)	S	S	None	S

- Final heading: S
- Does the first step require a turn? Yes — Why? The robot starts facing North but needs to go East (column increases), so it must make a right turn.

Part 7: Robot Orientation Diagrams

- Scenario A: Facing North, need to go East → Turn: Right 90
- Scenario B: Facing East, need to go South → Turn: Right 90
- Scenario C: Facing West, need to go East → Turn: 180

- Scenario D: Facing South, need to go South → Turn: None

Part 8: Turn Logic in Code

```
right_turns = {"N": "E", "E": "S", "S": "W", "W": "N"}
left_turns  = {"N": "W", "W": "S", "S": "E", "E": "N"}

def get_turn(current_heading, needed_direction):
    if current_heading == needed_direction:
        return "NONE"
    elif right_turns[current_heading] == needed_direction:
        return "RIGHT"
    elif left_turns[current_heading] == needed_direction:
        return "LEFT"
    else:
        return "REVERSE"
```

Test your understanding:

1. `get_turn("E", "N")` returns "LEFT"
2. `get_turn("W", "W")` returns "NONE"
3. `get_turn("N", "S")` returns "REVERSE"

Reflection

Why we separate path planning from turning logic: Separating "where to go" from "how to get there" makes each piece simpler to understand, test, and debug independently. The Manhattan algorithm doesn't need to know anything about the robot's physical heading – it just plans coordinates. The turning logic doesn't need to know about destination planning – it just handles step-by-step orientation. This is called "separation of concerns" and it means you can test each part alone, fix bugs in one without breaking the other, and even reuse the Manhattan algorithm with a different robot that turns differently.